第 2 章練習　作業系統結構

本檔只含題目。請先自行建立狀態或變數表，再作答；解答在獨立檔案。

OS02-E01　API 與 system call

應用程式呼叫一個程式庫函式 `send_record()`，它可能寫入核心緩衝區並排入裝置佇列。

1. 哪些步驟一定發生在核心模式？
2. 程式庫回傳成功是否等於資料已持久？
3. 若核心從使用者指標複製資料，需驗證什麼？

OS02-E02　Microkernel 取捨

有人主張把所有檔案系統與網路服務移出核心，系統就會更安全。

1. 支持此主張的隔離理由是什麼？
2. 至少列出兩個可能成本或新風險。
3. 提出一種能降低風險而不假設架構必然安全的方法。

OS02-E03　開機信任鏈

某機器由韌體載入未簽章 boot loader，再由 boot loader 載入簽章核心。

1. 核心簽章能否阻止 boot loader 替換核心映像？
2. 要建立可驗證信任鏈，驗證應在何處進行？
3. 模組簽章還需搭配哪些控制？

OS02-E04　Short write與複製不變量

來源6000 bytes且不再修改，空目的檔案、buffer4096。read結果4096、1904、0；write正回傳依序1000、3096、1024、880，無並行writer。

1. 逐步列出來源與目的offset，計總read/write bytes。
2. 另考慮錯誤版本：每次read後只做一次write，明定兩個讀取區塊的第一次write各回1000與1024；會遺失多少bytes？不得把原本對3096-byte剩餘區間的回傳3096套到新的1904-byte請求。
3. 給一般迴圈不變量並說明short write與EINTR。
4. 成功fsync目的是否足以保證新pathname持久且atomic replace？

OS02-E05　Pipeline端點與EOF

pipe回fd3讀/fd4寫，shell fork cat與wc兩個children。cat將fd4 dup2成stdout，wc將fd3 dup2成stdin。

1. 列exec後各process應保留的pipe引用。
2. cat已寫6000bytes並退出，wc仍讀不到EOF的一個具體原因？
3. cat對不存在a回非零，wc輸出0且exit0是否矛盾？
4. 兩次fork再兩次exec會產生四個child嗎？

OS02-E06　ABI範圍與TOCTOU

模型ABI有24-byte argument block與8-byte pathname。32-bit size_t，user上界U=C0000000，非空byte ranges，element size4。

1. 算copy bytes，並區別API/ABI/implementation。
2. 令p=0xBFFFFF00，n是byte length。分別判n=0x100與n=0x101；寫避免溢位的條件。
3. count=40000001(hex)乘4的正確值與截斷錯誤？
4. 先check VMA再直接dereference為何不足？

OS02-E07　Policy切換與layer依賴

單CPU A/B/C burst5/3/1ms全t0，RR初序A,B,C，無I/O、切換成本0。另有pager→driver→scheduler→pager依賴。

1. 算q1與q4的completion與response。
2. 平均response與task switch次數？
3. Mechanism與policy各有哪些？
4. 為何依賴環不能放進嚴格layer？如何具體拆解？

OS02-E08　Trace、尾端延遲與race反例

request在t0開始，t1block，t8ready，t9running，t10return。另一實驗兩threads各對x=0做一次非atomic increment。latency樣本為1,1,1,1,1,1,1,100ms。

1. 將10ms分成CPU、blocked、ready等待。
2. 算mean與nearest-rank p99。
3. 給lost update交錯，說明printf為何可能掩蓋。
4. 只對兩次write上mutex可修嗎？

OS02-E09　Root依賴閉包與信任鏈

root在storage S、filesystem F。K內建其餘必需元件，modules僅可從initramfs或已mount的root取得。另有firmware→loader→kernel的簽章policy。

1. S只放root、F內建，能否mount root？
2. S與F都在initramfs，為何仍非boot成功證明？
3. Kernel簽章由不受信任loader自行驗，足夠嗎？
4. 測量digest、合法簽章與anti-rollback各回答什麼？

OS02-E10　輪詢、中斷與 DMA

裝置於 t=4.2 ms 完成；poll 在 0.5、1.0、⋯ ms 開始，每次占 CPU 0.01 ms，開始時間精確且狀態立即可見。另一方案的 interrupt 立即交付，handler 占 CPU 0.04 ms。

1. 求定時 polling 第一次發現完成的時刻、查詢次數、偵測延遲及 CPU 查詢時間。
2. 求 interrupt handler 結束時刻與 CPU 處理時間。若改為 0–4.2 ms 一直 busy-poll，CPU 使用時間多少？
3. DMA 與通知方式如何搭配？為何不能在尚未完成交接前重用 DMA buffer？

OS02-E11　完成、就緒與執行

延續上一題 interrupt 方案，假設 handler 結束時使 P ready，而 P 在 t=4.34 ms 首次 running。算 P 的 ready 等待；說明為何裝置完成、handler 完成、P running 與 system call 返回是不同事件。若 handler 在 CPU 已處於 kernel mode 時收到中斷，能否直接說必有 user→kernel transition？

參考資料與署名

Lin Cing，《作業系統》手寫筆記。 Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapters 1–2. write(2); fsync(2); pipe(7); Linux 3.2 init/main.c.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study